

ePGD DL & GENAI: Assignment (Apr 2026)

Sem II - Ashish Trivedi

April 2026

Part B: Recurrent Neural Networks (Sentiment Classification)

Dataset Information

Source: IMDB Large Movie Review Dataset ¹

Problem Description

Consider the task of sentiment classification using a standard natural language processing dataset. The objective of this part is to study and compare the effectiveness of different recurrent neural network architectures, namely vanilla recurrent neural networks (RNNs), long short-term memory networks (LSTMs), and attention-augmented variants, for binary sentiment classification.

Use the IMDB Large Movie Review Dataset, which consists of English movie reviews annotated with binary sentiment labels (positive or negative). Each input sample is a sequence of tokens corresponding to a review, and the model output is a single sentiment label.

1 B1: Data Loading and Preprocessing

Data Acquisition

The `aclImdb.v1.tar.gz` dataset was downloaded from Stanford's website and extracted to the local environment. This dataset contains 50,000 movie reviews, evenly split between positive and negative sentiments (25,000 training, 25,000 testing).

Data Loading

Custom functions were used to load the positive and negative review text files into pandas DataFrames, `train_df` and `test_df`, each with `text` and `label` columns.

Text Preprocessing

A `preprocess_text` function was defined and applied to all review texts. This function performed the following critical steps:

- **Normalization:** Converted text to lowercase and removed HTML tags and punctuation.
- **Tokenization:** Broke down the text into individual words (tokens).
- **Redundancy Reduction & Stop-word Removal:** Applied lemmatization to reduce words to their base form and removed common English stop words (e.g., 'the', 'is', 'a') to focus on more meaningful terms.

Summary

This section focused on preparing the IMDB movie review dataset for sentiment classification. This preprocessing ensures that the text data is clean, consistent, and ready for numerical representation in subsequent modeling steps.

¹<https://ai.stanford.edu/~amaas/data/sentiment/>

2 B2: Token Mapping and Embedding Preparation

Data Splitting

The initial `train_df` was further split into training and validation sets (80/20 ratio) to monitor model performance during training and prevent overfitting. The `test_df` was kept separate for final evaluation.

Vocabulary Construction & Tokenization

- A `Tokenizer` from Keras was fitted on all processed text (train, validation, and test) to build a comprehensive vocabulary.
- An `<unk>` token was used to handle out-of-vocabulary words.
- The total vocabulary size identified was 203,436 unique tokens.
- All text reviews were converted into integer sequences based on this tokenizer.

Sequence Padding

The maximum sequence length observed was 1420, with an average of 118 words per review. A fixed sequence length of 250 was chosen for padding, meaning reviews longer than 250 tokens were truncated, and shorter ones were padded with zeros. This results in padded input arrays like `X_train_padded` with shape (20000, 250).

Embedding Preparations

- **One-Hot Encoding (Conceptual):** Explained as a high-dimensional, sparse representation, generally not practical for direct use in RNNs.
- **Word2Vec Embeddings:** A Word2Vec model was trained directly on the processed text corpus, resulting in an embedding dimension of 100. An `embedding_matrix_w2v` of shape (203436, 100) was created, mapping each word in our vocabulary to its Word2Vec vector.
- **GloVe Embeddings:** The pre-trained GloVe `glove.6B.100d.txt` embeddings were downloaded and loaded. An `embedding_matrix_glove` of shape (203436, 100) was constructed, aligning GloVe vectors with our vocabulary. Words not found in GloVe were zero-initialized.

3 B3: Construct Vanilla Recurrent Neural Network (RNN)

Define Hyperparameters

We define the number of recurrent layers ($L = 2$) and the hidden dimension $h = 128$ for the model.

Model Architecture

- **Embedding Layer:** Converts integer-encoded words into dense, fixed-size vectors. Initialized with our pre-trained `embedding_matrix_w2v`. We set `trainable=False` to keep them fixed during training.
- **SimpleRNN Layer:** The core recurrent component. Processes sequences one element at a time, maintaining an internal hidden state. The value $h = 128$ determines the dimensionality of the hidden state.
- **Dense Output Layer:** A single unit with `sigmoid` activation function to output a probability between 0 and 1.

4 B4: Train the RNN Model

The model was compiled with the `adam` optimizer and `binary_crossentropy` loss function. Training was run for 5 epochs with a batch size of 32.

5 B5: Construct and Train the LSTM Model

An LSTM model was constructed with similar hyperparameters, replacing the SimpleRNN with LSTM layers. The LSTM uses internal mechanisms (gates) to regulate the flow of information, effectively mitigating the vanishing gradient problem.

6 B6: Attention-Augmented LSTM Model

The model builds upon the LSTM architecture by incorporating an attention mechanism. The attention layer dynamically learns to weigh the importance of different hidden states produced by the Bidirectional LSTM, allowing the model to focus on the most relevant parts of the text for sentiment classification.

7 B7 Comparative Analysis: Models vs. Embedding Parameters and Performance

Table 1: Model Performance and Parameter Comparison

Model Type	** E.T.	** L	Test Accuracy	Test Loss	Total Params	Trainable Params
Vanilla RNN	Word2Vec	2	54.32 %	68.87 %	20,405,937	62,337
LSTM	Word2Vec	2	84.98%	38.08%	20,592,561	248,961
LSTM	GloVe	2	60.52%	68.26%	20,592,561	248,961
Atten.-Aug. LSTM	Word2Vec	2	87.79%	30.17%	20,972,850	629,250

** E.T. - Embedding Type

** L - Layers

Overall Conclusions:

- Vanilla RNN's Limitations:** The basic *SimpleRNN* model struggles significantly with long-range dependencies, resulting in accuracy barely above random chance. This reinforces the theoretical understanding of vanishing gradients in simpler RNN architectures.
- LSTM's Superiority:** Replacing 'SimpleRNN' with 'LSTM' layers dramatically improves performance. The LSTM's gated memory cells effectively address the vanishing gradient problem, enabling the model to retain crucial information over extended sequences. The LSTM with Word2Vec embeddings achieved a strong accuracy of 85.
- Embedding Choice Matters:** While both Word2Vec (trained on IMDB) and GloVe (general-purpose) are popular pre-trained embeddings, the Word2Vec embeddings (derived from the specific IMDB dataset) yielded substantially better results for the LSTM model. This highlights the importance of domain-specific embeddings or fine-tuning general embeddings for the target task.
- Attention Mechanism's Impact:** The addition of an attention mechanism to the Bidirectional LSTM model further boosted performance, achieving the highest accuracy of 88.
- Parameter Analysis:** * The vast majority of 'Total Parameters' for all models come from the non-trainable embedding layer, reflecting the size of the vocabulary and embedding dimension. * 'Trainable Parameters' increase with model complexity: Vanilla RNN < LSTM < Attention-Augmented LSTM. This correlation generally aligns with the observed performance improvements, as more complex models have a greater capacity to learn intricate patterns.

Final Recommendation:

For sentiment classification on this dataset, the **Attention-Augmented Bidirectional LSTM with Word2Vec embeddings** offers the best balance of model complexity and performance, effectively addressing the challenges of long-range dependencies and enhancing the model's interpretability by focusing on critical sentiment cues.